

Trustworthy AI Assignment 3

Hanyoung An (2021440073)

Abstract—This report summarizes neural network verification experiments using Marabou, an SMT-based verification framework. First, I reviewed the official Marabou resources directory to understand the provided model formats, benchmark networks, and verification query examples. Then, I applied Marabou to external ONNX models generated in this repository. The main verification property is local robustness under an L_∞ input perturbation. The results show that Marabou can provide formal answers for small, well-defined verification queries, but its runtime increases rapidly as the number of ReLU nodes and the perturbation radius increase.

I. INTRODUCTION

Neural network verification aims to formally prove whether a trained model satisfies a desired property. Unlike empirical testing, which can only show that no failure was found among tested samples, formal verification can prove that no counterexample exists inside a specified input region. In this assignment, I used Marabou, an SMT-based neural network verification framework, to check local robustness properties of small neural networks [1].

The assignment consists of two parts. In Problem 1, I explored the official Marabou `resources/` directory. The directory contains benchmark networks, input specifications, and example verification queries. In Problem 2, I selected external models that are not from the Marabou resources directory, exported them to ONNX format, and used Marabou to verify local robustness around selected input samples.

II. MAIN DISCUSSION

A. Marabou Resources Exploration

The Marabou resources directory provides several benchmark models and query formats. The main model formats are `.nnnet` and `.onnx`, while verification properties are often written in formats such as `.vnnlib`, text-based property files, or Marabou-specific query files. Table I summarizes representative resources found in the directory, including ACAS Xu networks, MNIST fully connected networks, CIFAR-10 ONNX classifiers, layer-level ONNX test models, and VNNLib property files.

TABLE I: Summary of Marabou resource examples.

Category	Example	Format	Purpose
ACAS Xu	ACASXu_v2a	.nnnet, .onnx	Collision avoidance
MNIST MLP	mnist2x256	.nnnet, .onnx	Robustness benchmark
CIFAR-10	cifar_base	.onnx	CNN verification
Layer zoo	ReLU, Conv, MaxPool	.onnx	Parser testing
VNNLib	acasxu_prop1	.vnnlib	Property specification

From this exploration, I found that ONNX is the most practical format for custom models because it can be exported from common Python machine learning workflows. However,

complex modern architectures may contain unsupported operators, so I used simple fully connected ReLU models in Problem 2.

B. External Models

I generated three external ONNX models: one Iris MLP and two MNIST MLPs. These models are not copied from the Marabou resources directory. As shown in Table II, the models differ in input dimension and hidden layer width. The Iris model has only four input features, so it is useful for checking whether the verification pipeline works correctly. The two MNIST models are used to observe how verification time changes when the hidden layer width increases.

TABLE II: External models used for verification.

Model	Dataset	Architecture	Input space
Iris MLP	Iris	$4 \rightarrow 8 \rightarrow 3$	Raw cm features
MNIST narrow	MNIST	$784 \rightarrow 32 \rightarrow 10$	Raw pixels $[0, 255]$
MNIST wide	MNIST	$784 \rightarrow 128 \rightarrow 10$	Raw pixels $[0, 255]$

For compatibility with Marabou, preprocessing was folded into the first affine layer. In the Iris model, the StandardScaler transformation was absorbed into the first layer. In the MNIST models, the pixel normalization operation $x/255$ was also folded into the first layer. Therefore, all exported ONNX models share a simple graph structure:

$$\text{MatMul} \rightarrow \text{Add} \rightarrow \text{ReLU} \rightarrow \text{MatMul} \rightarrow \text{Add}.$$

C. Verification Query

The target property is local robustness. For a fixed input sample x , Marabou checks whether all perturbed inputs x' inside an L_∞ ball preserve the original predicted class. Each input variable is bounded as

$$\max(0, x_i - \epsilon) \leq x'_i \leq x_i + \epsilon.$$

Instead of directly proving the safe condition, the implementation asks Marabou to find an unsafe counterexample. For each target class t different from the predicted class p , the following constraint is added:

$$y_t \geq y_p + 10^{-6}.$$

Thus, SAT means that an adversarial counterexample exists inside the perturbation region, while UNSAT means that no such counterexample exists for the tested input and radius. TIMEOUT means that Marabou could not finish within the given time limit, so robustness is not proven.

D. Verification Results

Table III shows the observed verification results. For the Iris model, small perturbation radii were solved almost instantly. At $\epsilon = 1.0$, Marabou found a counterexample, which means that the local robustness property does not hold for that radius. For the MNIST narrow model, $\epsilon = 1.0$ and $\epsilon = 5.0$ were verified as UNSAT, while $\epsilon = 20.0$ produced a SAT result. For the MNIST wide model, the runtime increased significantly. At $\epsilon = 10.0$, one target class produced a counterexample, but other target queries reached the timeout.

TABLE III: Verification results.

Model	ϵ	Status	Runtime
Iris MLP	0.1	UNSAT	0.002s
Iris MLP	0.5	UNSAT	0.002s
Iris MLP	1.0	SAT	0.004s
MNIST narrow	1.0	UNSAT	0.33s
MNIST narrow	5.0	UNSAT	0.54s
MNIST narrow	20.0	SAT	1.01s
MNIST wide	1.0	UNSAT	1.66s
MNIST wide	5.0	UNSAT	9.43s
MNIST wide	10.0	SAT + TIMEOUT	145.9s

These results demonstrate two important observations. First, increasing ϵ makes the property harder to satisfy because the allowed input region becomes larger. Second, increasing the number of ReLU hidden units makes the verification problem more expensive. The Iris model has only 8 ReLU nodes and is solved almost instantly. The MNIST narrow model has 32 ReLU nodes and remains practical. The MNIST wide model has 128 ReLU nodes and already shows a large runtime increase with partial timeouts.

E. Discussion

The main strength of Marabou is that it provides formal SAT/UNSAT answers for explicitly encoded properties. In this experiment, an UNSAT result means that no adversarial counterexample exists within the specified L_∞ box for the selected sample and target classes. This is stronger than simply running an empirical adversarial attack.

However, the guarantee is local and property-specific. An UNSAT result for one input sample and one perturbation radius does not imply global robustness of the whole model. Also, scalability is a clear limitation. Even a simple one-hidden-layer MNIST model became much harder to verify when the hidden layer size increased from 32 to 128. Another practical issue is model compatibility. To make the ONNX models easy for Marabou to parse, I had to keep the graph simple and fold preprocessing operations into affine layers.

III. CONCLUSION

In this assignment, I reviewed the Marabou resources directory and applied Marabou to external ONNX models. The experiments confirmed the expected interpretation of SAT,

UNSAT, and TIMEOUT results for local robustness verification. The results also showed that formal verification is useful for small, well-defined neural network properties, but the cost increases rapidly with model size and perturbation radius. Overall, Marabou is valuable for understanding neural network verification and for checking local safety properties, while large-scale practical deployment requires careful model simplification and property selection.

REFERENCES

- [1] G. Katz, D. A. Huang, D. Ibeling, K. Julian, C. Lazarus, R. Lim, P. Shah, S. Thakoor, H. Wu, A. Zeljić, D. L. Dill, M. J. Kochenderfer, and C. Barrett, “The marabou framework for verification and analysis of deep neural networks,” in *Computer Aided Verification*. Springer, 2019, pp. 443–452.